CS 473 (Spring 2025) Yiming Qin (yimingq3@illinois.edu)

Problem 5.2

Solution:

(a) Split sets into two halves L and R, compute all $g_k (0 \le k \le \ell)$ using divide-and-conquer. For all possible $g_i(L)$ and $g_j(R)$ s.t. $i + j = k$, compute $g_i(L) + g_j(R)$ using FFT. Take the union of all such sets into $g_k(S_1, \ldots, S_\ell)$.

Algorithm 1

```
1: function DisjointSubsetSum(S)
2:     ℓ = length of S
3:     if ℓ = 0 then
4:         return φ
5:     L = S_1, S_2, ..., S_⌊ℓ/2⌋
6:     R = S_⌊ℓ/2⌋+1, ..., S_ℓ
7:     G_L = DisjointSubsetSum(L)
8:     G_R = DisjointSubsetSum(R)
9:     Initialize array G of length ℓ + 1, each element is φ
10:    for i = 0 to ⌊ℓ/2⌋ do
11:        for j = 0 to ⌊ℓ/2⌋ + 1 do
12:            if i + j ≤ ℓ then
13:                Compute g = g_i(L) + g_j(R) by taking convolution of G_L[i] and G_R[j]
14:                G[i + j] = g ∪ G[i + j]
15:    return G
16: S = [S_1, S_2, ..., S_ℓ]
17: return DisjointSubsetSum(S)[k]
```

Time complexity analysis:

Steps 1 and 2 split the problem into 2 subproblems and solve them recursively. In step 3, both halves have $O(\ell)$ sets, so there are $O(\ell^2)$ pairs of (i, j) such that $i + j = k$. Since the largest number in any set is ℓN and $\ell \le N$, each set contains at most $O(\ell N)$ elements, computing each $g_i(L) + g_j(R)$ using FFT takes $O(\ell N \log N)$ time. Thus the total time complexity is:

$$T(\ell, N) = 2T(\ell/2, N) + O(\ell^3 N \log N) = O(\ell^3 N \log N). \tag{1}$$

(b) Randomly partition S in to ℓ disjoint sets. For any specific set of k elements $(a_1, \ldots, a_k)$, the sum of the set s appears in $g_k(S)$ if each element is properly assigned to different disjoint subset. Since s can be computed by multiple sets, the possibility that $f_k(S)$ contains s is:

$$q = \Pr[s \in g_k(S)] \ge \prod_{i=0}^{k-1} \frac{\ell - i}{\ell}. \tag{2}$$

Let $\ell = k$, $q = \frac{k!}{k^k}$ is a constant for a particular k. Assume we randomly partition S for c times and compute $g_{k,i}(S)$ for the ith partition, the possibility that s does not appear in any

$g_{k,i}(S)$ is:

$$\Pr\left[\bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] = (1-q)^c \tag{3}$$

There are at most kN elements in $f_k(S)$, accroding to Union Bound, the possibility that the union of all $g_{k,i}(S)$ does not contain all possible elements is:

$$\Pr\left[\bigcup_{s\in f_x(S)} \bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] \leq kN \cdot \Pr\left[\bigcap_{i=1}^{c} s \notin g_{k,i}(S)\right] = kN(1-q)^c \tag{4}$$

Rewrite $kN(1-p)^c \leq 1/N$, we obtain,

$$c \geq \frac{2logN}{|log(1-q)|} \tag{5}$$

Algorithm 2

```
c = ⌈2logN⌉
for i = 1 to c do
    Randomly partition S into k subsets
    Compute g_{k,i}(S) using algorithm in (a)
return ⋃_{i=1}^{c} g_{k,i}(S)
```

Let $c = O(logN)$, overall error probability is at most $1/N$, total time complexity is $O(k^3 N log^2 N)$.